

PROGRAMA INTEGRAL DE FORMACIÓN EN PYTHON

CURSO 1: FUNDAMENTOS BÁSICOS DE PYTHON (NIVEL 1
(PRINCIPIANTES))

Clase 08: Integración Total & Proyecto Integrador

«El Casco de Seguridad y Salir a Rodar en Bici»

Consolidación de todo el Curso 1 en una aplicación completa: arquitectura CRUD (Create, Read, Update, Delete) para un gestor interactivo de tareas en consola.

Instructor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Nivel del Curso: Principiante a Intermedio | **Python:** 3.10+ | **Licencia:** MIT

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre **wisrovi SUITE** en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Estructura Pedagógica de Cada Guía

Cada documento de esta serie está diseñado rigurosamente bajo el estándar de ingeniería de software profesional:

- **Fundamento Conceptual y Metáfora Intuitiva:** Anclaje visual del concepto abstracto a la realidad.
- **Diagrama de Flujo y Arquitectura Mental:** Representación visual de cómo fluye la información.
- **Código de Nivel Profesional:** Sintaxis limpia, comentarios línea a línea y tipado estático (PEP 484).
- **Patrones y Gotchas:** Errores comunes que cometen los desarrolladores y cómo evitarlos.

Índice General de la Sesión

Sección	Contenido y Enfoque Temático	Página
Capítulo 1	Fundamentos y Metáfora Conceptual: Arquitectura del Proyecto Integrador: Gestor de Tareas	Pág. 4
Capítulo 2	Diagrama de Flujo y Arquitectura: Diagrama de Arquitectura de la Aplicación CLI	Pág. 5
Capítulo 3	Implementación Práctica en Python: Implementación del Task Manager CLI	Pág. 6
Capítulo 4	Patrones Avanzados, Gotchas y Debugging: Buenas Prácticas para Proyectos Reales	Pág. 7
Cierre	Conclusiones, Notas del Mentor y Agradecimiento	Pág. 8
Anexos	Bibliografía Oficial y Enlaces de Profundización	Pág. 9

Objetivos de Aprendizaje (Competencias Clave)



Competencia Conceptual

Comprender cómo se interconectan todos los pilares del lenguaje para crear una aplicación funcional y resiliente.



Competencia Práctica

Construir de principio a fin un sistema de gestión en terminal con menús interactivos, validaciones y persistencia conceptual.

Requisitos Previos y Entorno Recomendado

Para aprovechar al máximo esta sesión se recomienda tener instalado Python 3.10 o superior, Visual Studio Code con la extensión oficial de Python configurada, y haber completado las lecturas y ejercicios de las sesiones anteriores de la ruta formativa.

1. Arquitectura del Proyecto Integrador: Gestor de Tareas

Llegó el momento de unir todas las piezas: variables, condicionales, bucles, listas, diccionarios y funciones trabajando en armonía.

☀ **Metáfora Central: El Casco de Seguridad y Salir a Rodar en Bici**

Hasta ahora hemos practicado el equilibrio con las rueditas de entrenamiento. Hoy nos quitamos las rueditas, nos ponemos el casco de seguridad y salimos a rodar en la bicicleta por nosotros mismos en el mundo real.

Principios Teóricos y Modelo Mental

Patrón de Menú Principal: Un bucle infinito `while True` mantiene viva la aplicación hasta que el usuario decida salir explícitamente.

Capa de Datos: Una lista de diccionarios en memoria actúa como la base de datos temporal de la aplicación.

⚡ **Regla de Oro en Python**

Separa la presentación (`print`, `input`) de la lógica de negocio (las funciones que agregan, buscan y transforman datos).

2. Diagrama de Arquitectura de la Aplicación CLI

Interacción entre la capa de interfaz de consola, el enrutador de comandos y el modelo de datos.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Bucle principal muestra el menú de opciones (1. Agregar, 2. Listar, 3. Completar, 4. Salir).	Esperando opción del usuario
2. Evaluación	Enrutador if/elif invoca la función específica según la opción elegida.	Despacho a función modular
3. Transformación	La función ejecuta la operación CRUD sobre la lista de tareas en memoria.	Actualización del estado
4. Retorno / Salida	Se muestra retroalimentación visual al usuario y se reinicia el ciclo del menú.	Ciclo listo para nueva orden

Visualización Mental

Esta arquitectura modular en consola es idéntica en concepto a los controladores y servicios de las APIs web modernas.

3. Implementación del Task Manager CLI

Estructura modular del proyecto integrador con funciones CRUD completas:

main.py (Python 3.10+)

UTF-8 • PEP 8 Compliant

```
tareas: list[dict] = []

def agregar_tarea(titulo: str) -> None:
    nueva_tarea = {"id": len(tareas) + 1, "titulo": titulo, "completada": False}
    tareas.append(nueva_tarea)
    print(f"✅ Tarea #{nueva_tarea['id']} agregada con éxito.")

def listar_tareas() -> None:
    if not tareas:
        print("📭 No hay tareas registradas.")
        return
    for t in tareas:
        estado = "✅ [LISTA]" if t["completada"] else "⌚ [PENDIENTE]"
        print(f"#{t['id']} - {t['titulo']} {estado}")

def completar_tarea(id_tarea: int) -> None:
    for t in tareas:
        if t["id"] == id_tarea:
            t["completada"] = True
            print(f"🔄 Tarea #{id_tarea} marcada como completada.")
            return
    print("❌ ID de tarea no encontrado.")
```

Análisis del Código Fuente

Sistema modular que implementa el ciclo CRUD completo, demostrando el dominio integral de las estructuras de datos y funciones.

4. Buenas Prácticas para Proyectos Reales

Reglas de oro para dar el salto de principiante a desarrollador junior estructurado:

⚠ Gotcha Frecuente (Trampa de Principiante)

Escribir código espagueti con cientos de líneas sin funciones y mezclando variables globales descontroladas.

Patrón Recomendado vs Antipatrón

✗ Antipatrón / Mal Código

```
# Código monolítico sin funciones ni modularidad
while True:
    op = input()
    # 300 líneas de if/else anidados sin
    separación
```

✓ Patrón Pythonic / Correcto

```
# Código desacoplado
def main():
    while True:
        mostrar_menu()
        procesar_opcion()
```

🛡 Consejo de Resiliencia en Producción

Encapsula siempre el punto de entrada de tu programa dentro de `if __name__ == '__main__': main()`.

5. Resumen Ejecutivo y Conclusiones

¡Felicitaciones! Has completado con éxito el Curso 1 de Fundamentos de Python. Has pasado de cero a programador activo.



Logro Alcanzado en esta Clase

Creación y comprensión integral de tu primera aplicación de software estructurada en Python.

Notas del Instructor y Sigüientes Pasos

En el Curso 2 daremos el salto a Algoritmos Avanzados, Notación Big-O, Pilas, Colas, Búsqueda Binaria y Optimización.



Mensaje de Agradecimiento

Muchas gracias por tu entusiasmo, disciplina y dedicación al participar en este programa formativo. La programación es un superpoder que transforma vidas cuando se ejerce con constancia y curiosidad. ¡Nos vemos en la próxima sesión para seguir construyendo juntos!

«El código más elegante es aquel que cualquiera puede entender y nadie necesita reescribir.»

6. Fuentes Bibliográficas y Recursos de Estudio

Para profundizar y consolidar los conocimientos adquiridos en esta clase, consulta las siguientes referencias:

Recurso / Fuente	Descripción Temática	Enlace Oficial
Documentación Oficial de Python	Referencia canónica del lenguaje y librería estándar	docs.python.org/3/
PEP 8 - Style Guide for Python	Guía oficial de estilo, formato e indentación	peps.python.org
Real Python Tutorials	Artículos técnicos y patrones de desarrollo moderno	realpython.com
Python Type Checking (PEP 484)	Anotaciones de tipo y análisis estático en Python	docs.python.org/typing
Suite Open Source wisrovi	Paquetes Python para orquestación y alto rendimiento	github.com/wisrovi

Canales de Consulta y Comunidad

Si encuentras dificultades al replicar el código o tienes dudas sobre la arquitectura de los ejercicios, no dudes en abrir un Issue en el repositorio oficial del curso en GitHub o participar activamente en nuestras sesiones grupales de mentoría.



Desafío de Autoestudio Recomendado

Agrega la función para guardar y cargar las tareas en un archivo JSON en disco para tener persistencia real.